

AMiT
Arba Minch Institute of Technology
አርባ ምንጭ ቴክኖሎጂ ኢንስቲትዩት
Arba Minch University
አርባ ምንጭ ዩኒቨርሲቲ

Faculty of Computing and Software Engineering

Department of Software Engineering

SENG5232- Software Architecture and Design

Chapter 1: Introduction to Software Design

"Software architecture plays a pivotal role in the delivery of successful software yet it's frustratingly neglected by many teams."

Simon Brown

By Aliazar D.(MSc in Software Eng.) Email :- aliazar.deneke@amu.edu.et

Pre-Course Concepts



Brainstorming Questions

†1 Why does it take so long to get software finished?

†2 Why are development costs so high?

†3 Why can't we find all errors before we give the software to our customers?

†4 Why do we spend so much time and effort maintaining existing programs?

†5 Why do we continue to have difficulty in measuring progress as software is being developed and maintained?

Pre-Course Concepts/#0

† Software Engineering (SE)

- It is an application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software.

» The need of it arises because of software quality, higher rate of change in user requirements and environment on which the software is working.

» Large software, scalability, cost, dynamic nature and quality management

» Characteristics of good software must satisfy **operational**, **transitional** and **maintenance** grounds.

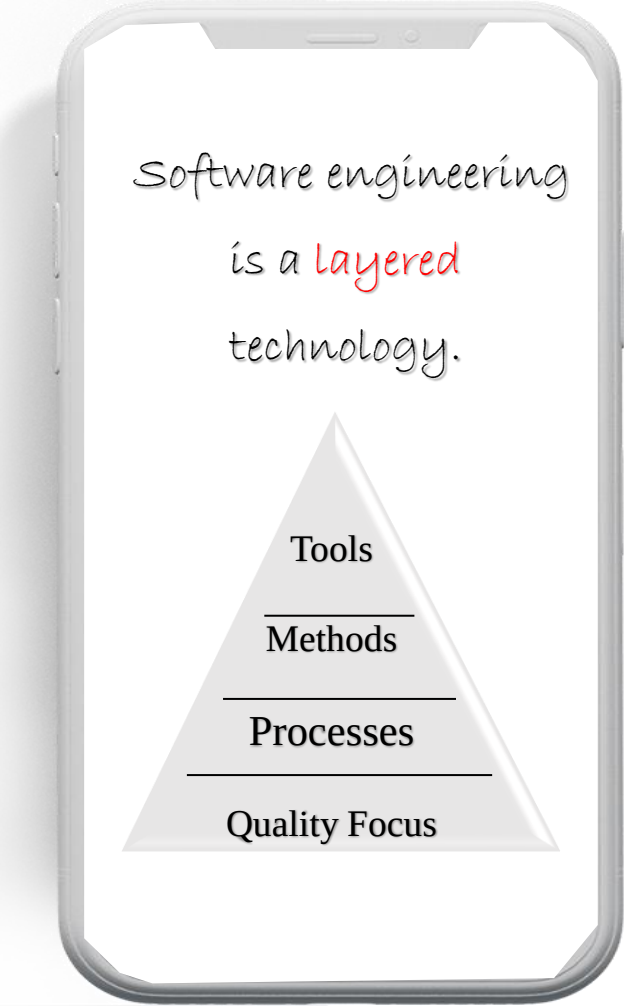
» **Operational:** **Reliability, Efficiency, Security, Scalability, Usability**

» **Transitional:** **Portability, Compatibility, Ease of Deployment, Data Migration**

» **Maintenance:** **Modularity, Documentation, Readability and Maintainability, Version Control, Bug Tracking**



Pre-Course Concepts/#0



- **Quality focus:** It defines the continuous process improvement principles of software. It provides integrity that means providing security to the software so that data can be accessed by only an authorized person, no outsider can access the data. It also focuses on maintainability and usability.
- **Process:** It is the foundation or base layer of software engineering. It is key that binds all the layers together which enables the development of software before the deadline or on time. Process defines a framework that must be established for the effective delivery of software engineering technology. The software process covers all the activities, actions, and tasks required to be carried out for software development.



- **Method:** During the process of software development the answers to all "how-to-do" questions are given by method. It has the information of all the tasks which includes communication, requirement analysis, design modeling, program construction, testing, and support.
- **Tools:** Software engineering tools provide a self-operating system for processes and methods. Tools are integrated which means information created by one tool can be used by another.

Pre-Course Concepts/#1

† **Requirements Engineering (RE)**

- Is the process of finding out, analyzing, documenting and checking services and operational constraints called **requirements for a computer based system.**
- Is the first software engineering activity, which translates the ideas or views into a requirements document.
- Is a subset of systems engineering that is concerned with systems as a whole including hardware, software and operational processes.
- But Good RE practices
  - » Can reduce the number of problems.
  - » Minimize their impact on the final system.

Pre-Course Concepts/#2

† **Software Development Life Cycles (SDLC)**

- Sometimes referred to as software life cycle.
- Is a *set of defined steps* in the development of a software product.
- Most scholars classify steps into 5, 6, or 7.
- Real world problems are the stimulus to which software serves as a response.
  - This implies that the starting step of software process is identifying the problem and the last step is delivering the software product.

† Most processes are either **plan-driven** or **agile**

- **Plan Driven Processes** are largely dependent on the plan prepared before the process starts.
  - *Examples: Waterfall Model, V-Model (Validation and Verification Model)*
- **Agile Processes** are basically changing accordingly with the changing user requirements.
  - *Examples: Scrum, Extreme Programming (XP), Feature-Driven Development (FDD)*

Pre-Course Concepts/#2

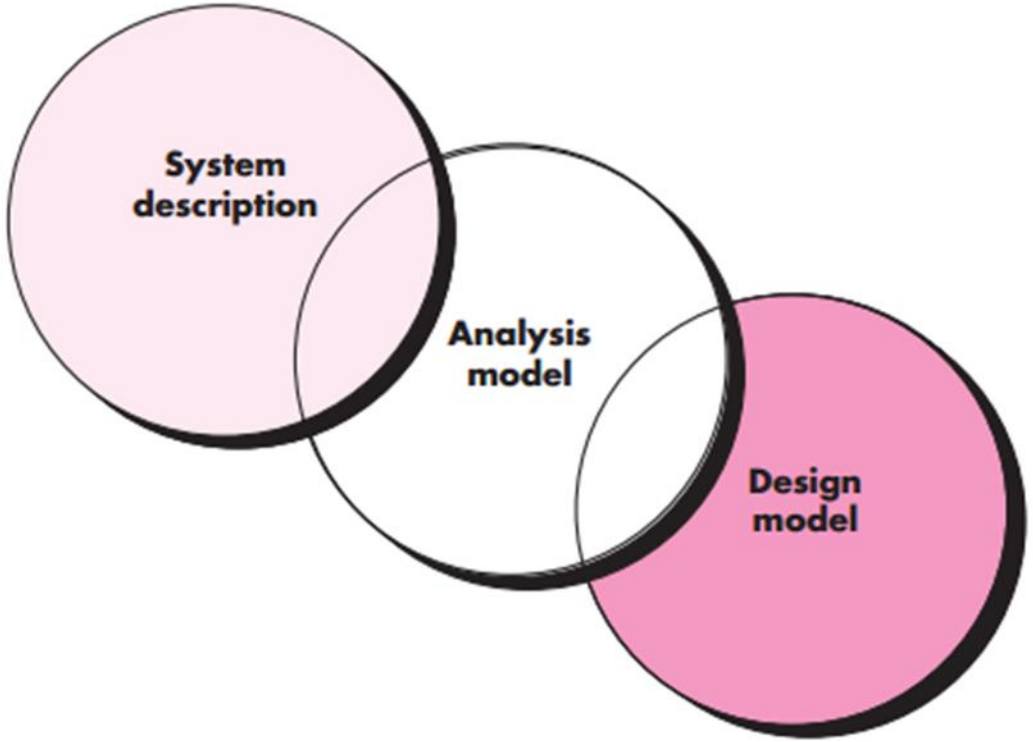
Software Development Life Cycles (SDLC) Steps

- **Requirements Elicitation**
  - **Is the process of gathering, documenting, and understanding the needs and expectations of stakeholders for a software project.**
- **Requirements Analysis** (e.g., *Context Analysis, Structured Analysis, OO Analysis, Behavioral Analysis*)
  - **Analyzing requirements and working towards a *conceptual* model *without* taking the target implementation technology into account.**
  - **Useful if the conceptual gap between requirements and implementation is large.**
  - **Part of requirements engineering (but may produce more than what is going to be part of the requirement spec).**
- **Design**
  - **Coming up with solution models *taking* the target implementation technology into account.**
- **Implementation**
- **Testing**
- **Deployment**
- **Maintenance and Support**
- **…**

Software Development Models

Models: Usually some sort of abstract language or pictures are used to express the software design.

- Used to provide small-scale plans in art and architecture.



Requirements Modeling (RM) /Analysis Model/

» It represents the user's requirements *by depicting the software* in three different domains:

- » ***Information domain:*** relates to the data and information that a software system manages and processes.
- » ***Functional domain:*** pertains to the specific functions or features that a software system provides to its users or stakeholders.
- » ***Behavioral domain:***. focuses on how the software system operates and behaves during execution.

Some of existing approaches of analysis modeling:

1. **Structured analysis**

- Considers data and the processes that transform the data as separate entities.
- Data objects are modeled in a way that defines their *attributes* and *relationships*.
- Processes that manipulate data objects are modeled in a manner that shows how they transform data as data objects flow through the system.

Requirements Modeling (RM) /Analysis Model/

2. **Object oriented analysis**
   - Focuses on the definition of classes and the manner in which they collaborate with one another to effect customer requirements.

Requirements Modeling (RM) /Analysis Model/

Analysis Rules of Thumb: created by Arlow and Neustadt

- The model should focus on requirements that are visible within the problem or business domain. The level of abstraction should be relatively high.
- Each element of the RM should add to an overall understanding of software requirements and provide insight into the information domain, function, and behavior of the system.
- **Delay** consideration of infrastructure and other nonfunctional models until design.
- Minimize coupling throughout the system. (the degree of interconnectedness or interdependence between different modules, components, or classes within a software system)
- Be *certain* that the *requirements model* provides value to all stakeholders. Each constituency has its own use for the model.
- Keep the model as simple as it can be.

Pre-Course Concepts/#3

Requirements Modeling (RM) /Analysis Model/

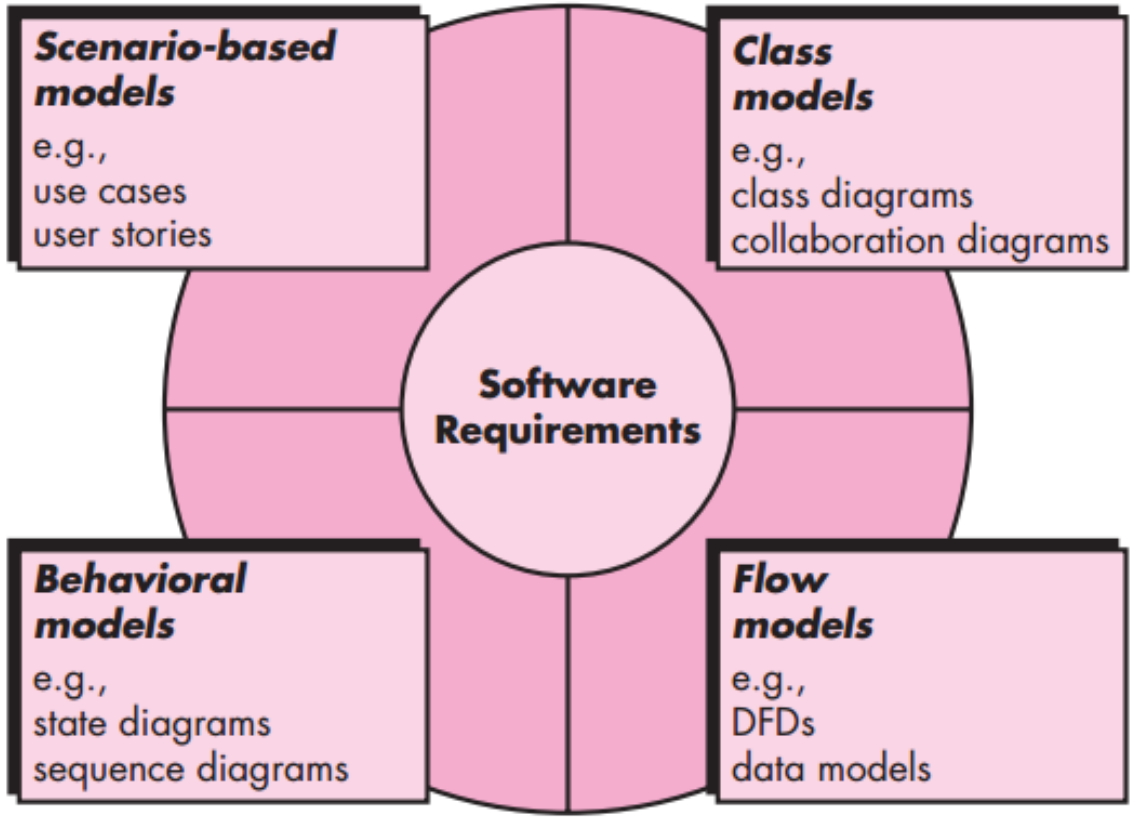
- The three primary objectives are:
  - Describe what the customers need
  - Establish basis for software design
  - Establish *set of requirements that are to be validated* after development
- All elements of the RM will be directly traceable to parts of the design model.
- Some design **invariably** occurs as part of analysis, and some analysis will be conducted during design.

Requirements Modeling (RM) /Analysis Model/

The basic types of models from requirements modeling are:

- **Scenario-based models** of requirements from the point of view of various system "actors".
- **Data models** that depict the information domain for the problem, the data objects and their interaction.
- **Class-oriented models** that represent object-oriented classes (attributes and operations) and the manner in which classes collaborate to achieve system requirements.
- **Flow-oriented models** that represent the ***functional elements*** of the system and how they **transform data** as it moves through the system.
- **Behavioral models** that depict how the software behaves as a consequence of external "events".

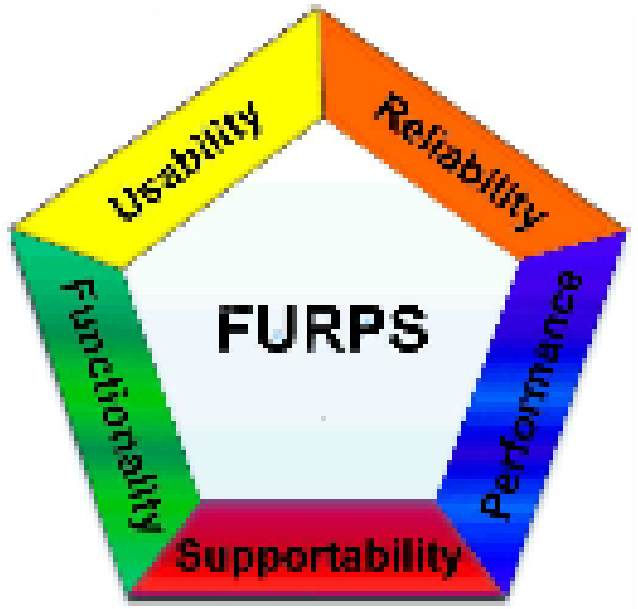
Requirements Modeling (RM) /Analysis Model/



Requirements Modeling (RM) /Analysis Model/

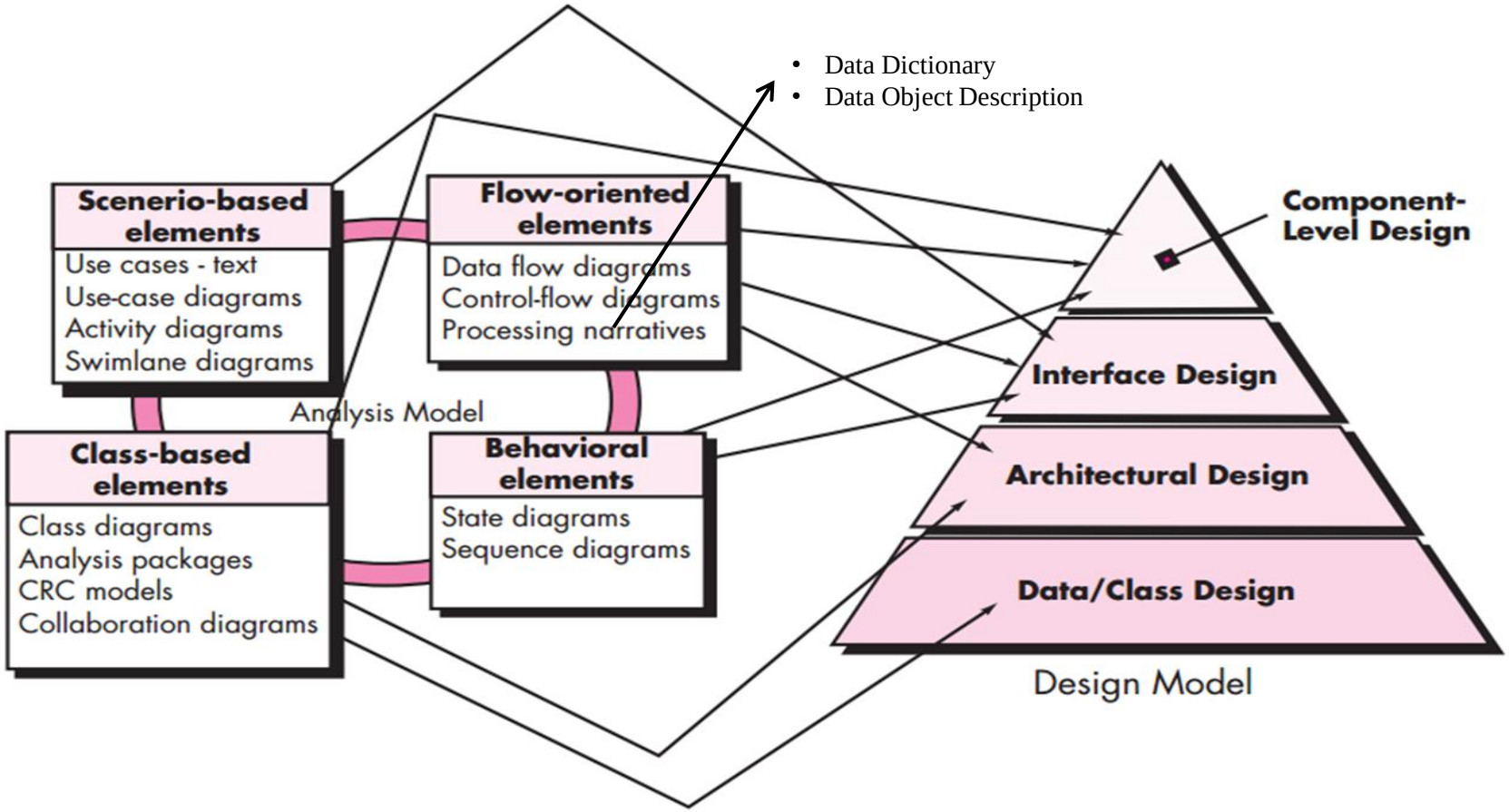
The ***requirements specification*** is modelled so that:

- The ***software designer translates*** it to architecture, component-level and interface designs.
- Along with the Requirements Specification Document(RSD), the developer and the customer can asses the software quality for **FURPS**.



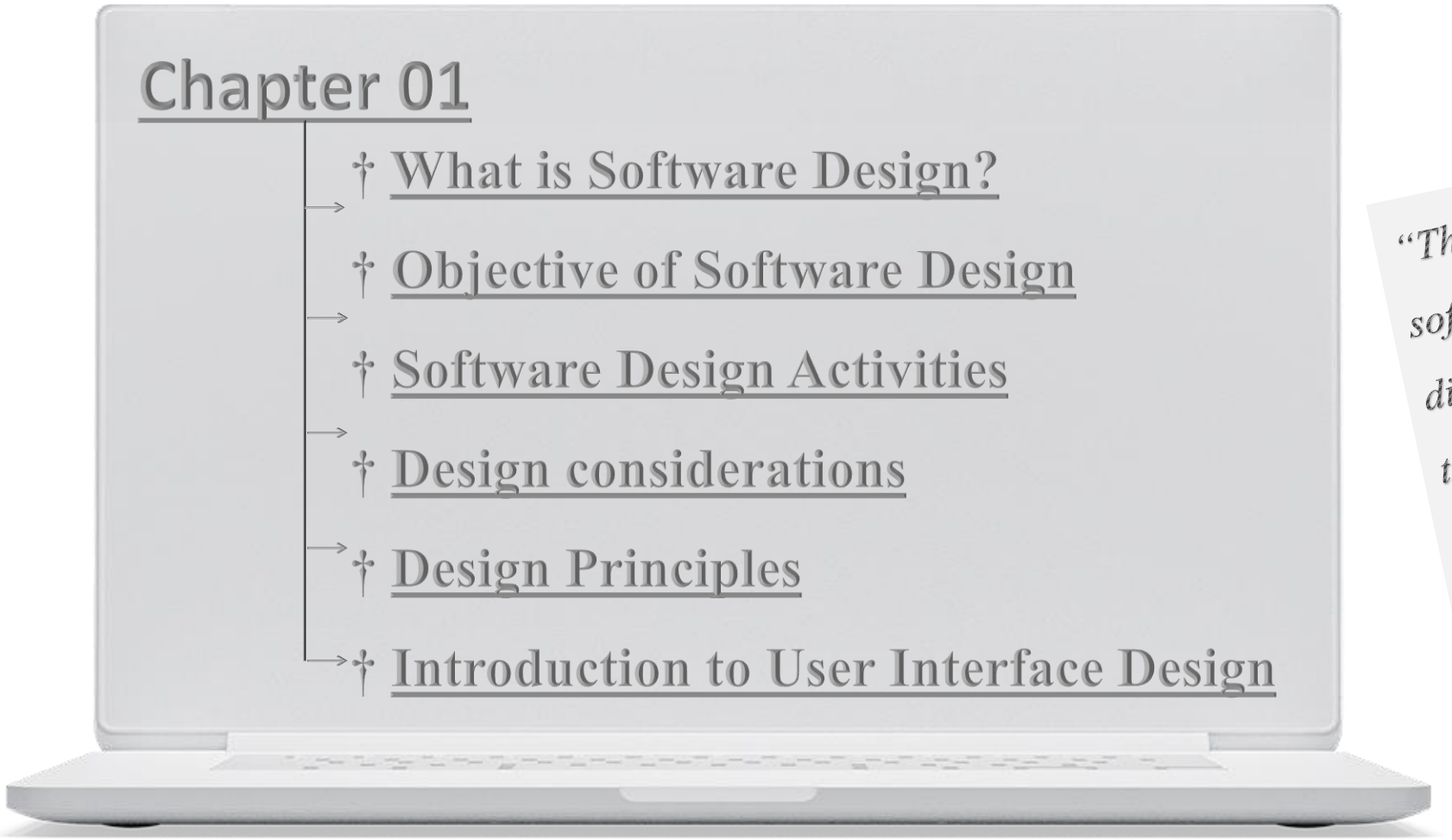
Pre-Course Concepts/#3

Translating analysis model to design model.



CRC - Class Responsibility Collaborator

Contents



"The beginning of wisdom for a software engineer is to recognize the difference between getting a program to work, and getting it right."

by M. A. Jackson

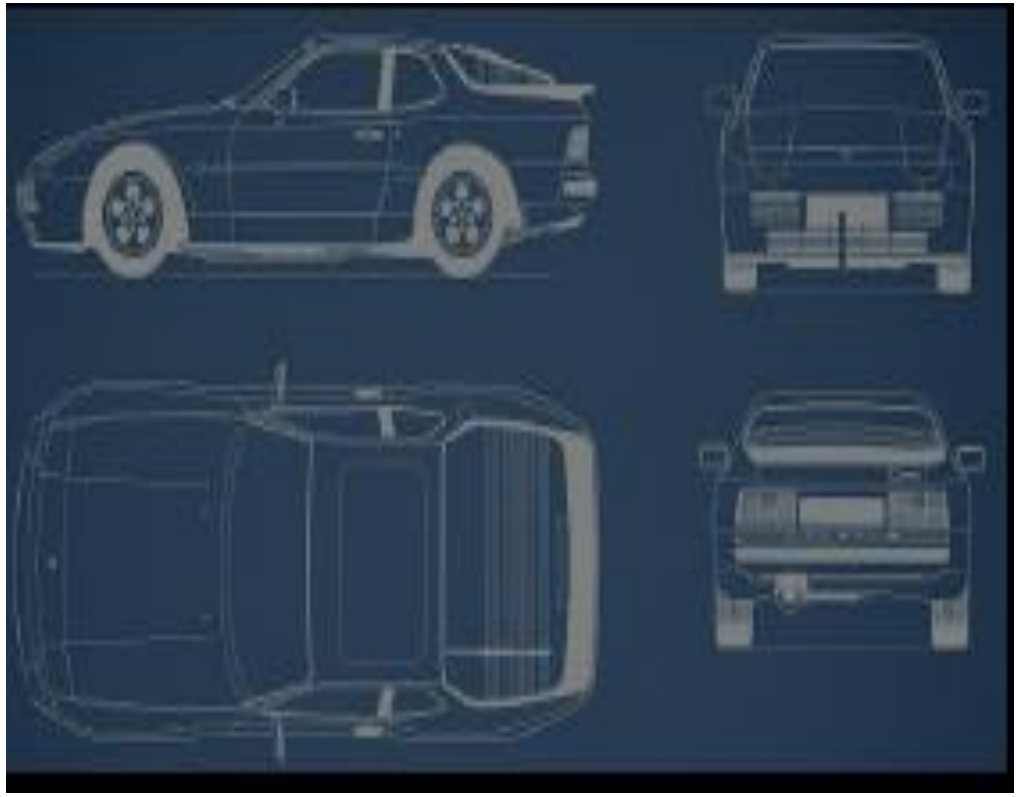
Brainstorming Questions

BS1: Can you define Software Architecture?

BS2: Can you define Software Design?

Software Architecture

- It is a representation of overall structure of an integrated system.
- The architecture of a system describes its major components, their relationships (structures), and how they interact with each other.
- It serves as a **blueprint for a system** that give us the **holistic view** (entire view) of a **software.**
- It provides an abstraction to manage the system complexity and establish a communication and coordination mechanism among components.
- In **Architecture**, nonfunctional decisions are cast and separated by the functional requirements. In Design, functional requirements are accomplished.
- It defines a **structured solution** to meet all the technical and operational requirements, while optimizing the common quality attributes like performance and security.



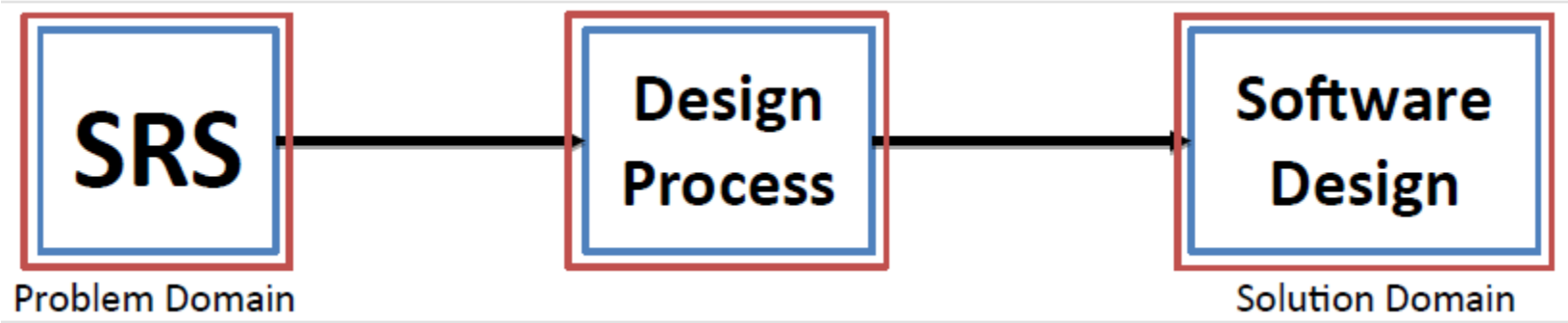
Software Design

What is Design?

› A **meaningful representation** of something to be built.

Software design provides a **design plan** that describes the elements of a system, how they fit, and work together to fulfill the requirement of the system.

- It's a **process** by which **requirements** are **translated** into **suitable form** for constructing a software.



Software architecture Vs. Software Design

It is about the complete architecture of the overall system.	It is about designing individual modules/components.
It defines the fundamental properties.	It defines the detailed properties.
It manages uncertainty.	It avoids uncertainty
It is more about the design of entire system.	Its more about on individual module/component
Its a plan which constrains software design to avoid known mistakes it achieves one organizations business and technology strategy.	Its considered as one initial phase of SDLC and it gives detailed idea to developers to implement consistent software.
Some of its pattern are micro service, server less and event driven	Some of its pattern are creational, structural and behavioral.
In general it refers to the process of creating high level structure of a software system.	In general it refers to the process of creating a specification of software artifact which will help to developers to implement the software
What we are building ?	How we are building?

Cont.

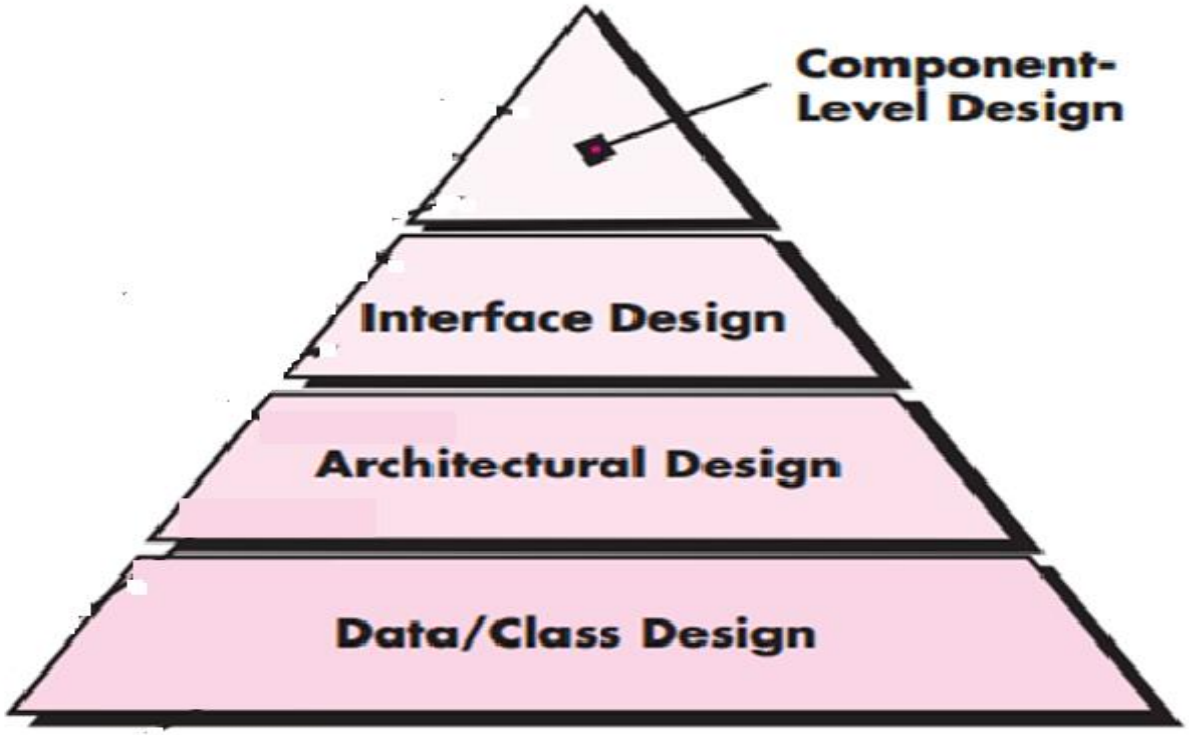
- Software Design is **the process to transform the user requirements into some suitable form**, which helps the programmer in software implementation.
- It can be traced to a customer's requirements and at the same time assessed for quality against a set of predefined criteria for "good design".
- To transform requirements into a working system, the designers must satisfy both the **customers** and the **system builders** of the development team. Hence the designers produce a "**conceptual design**" for the customers and a "**technical design**" for the system builders.
- A good conceptual design is written in the customer's language, contains no technical jargons, describes the ***functions of the system but independent of implementation*** and is linked to the requirements document.

Cont.

- According to IEEE **Software Design** is defined as the a process of defining, the components, architecture, interfaces, and other characteristics of a system or component and the result of that process.
- Software technical design focuses on four major areas of concern:
  - Data,
  - Architecture,
  - Interfaces and
  - Components.

Design Model

The software design hierarchy is shown below:



The Data Design

- The ***Data design*** *transforms the information domain model* created during analysis into the *data structures* that will be required to implement the software.
- *Data modeling* - The data model consists of ***three*** *interrelated pieces of information*: the data object, the attributes that describe the data object, and the relationships that connect the data objects to one another.
- ERD (Entity-Relationship Diagram) or UML(Unified Modeling Language) are used for data modeling.
- The data objects and relationships defined in the ERD and the detailed data content depicted in the data dictionary provide the basis for the ***data design activity.***
- Part of the data design ***may occur*** in conjunction with the *design of software architecture.*

Architectural Design

- The ***architectural design*** defines the relationship between the major structural elements of the software.
- It also *represents the constraints* that affect the way in which architectural design patterns can be applied. (Design patterns provide guidance on how to design and structure software components to meet specific requirements efficiently)
- It is very important because: It is an enabler for communication between all stakeholders.
- It highlights *early design decisions* that will have a profound impact on all the software engineering work that follows.
- It "constitutes a relatively small part, intellectually graspable model of how the system is structured and how its components work together".

Examples of AD decisions would be

- ***Will the system run on a single machine or be distributed over multiple CPUs?***
- ***Will outputs be stored in a file or in a database?***
- ***How will run-{}time errors be handled?***

User Interface Design

User interface design creates an effective communication between a human and a computer. Following a set of design principles, design identifies interface objects and actions, and then creates a screen layout that forms the basis for a *user interface prototype.*

- In order to build an ***effective user interface***, "all design should begin with the understanding of the intended users, including profiles of their age, sex, physical abilities, education, cultural or ethnic background, motivation, goals and personality".

Users belong to 3 broad categories namely –

- **Novices** (no syntactic knowledge and little semantic knowledge),
- **knowledgeable intermittent users** (reasonable semantic knowledge, but relatively low syntactic knowledge),
- **knowledgeable frequent users** (good semantic and syntactic users - "power-user syndromes")

Golden Rules of UI Design

- Place the user in control
  - Define interaction modes in a way that does not force the user into unnecessary or undesired actions.
  - Provide more flexible interaction.
  - Allow user interaction to be interruptible and undoable.
  - Streamline interaction as skill levels advance and allow the interaction to be customized.
  - Hide the technical internals from the casual user.
  - Design for interaction with objects that appear on the screen.
- Reduce the user's memory load
  - Reduce demand on short-term memory.
  - Establish meaningful defaults. Define shortcuts that are intuitive.
  - The visual layout of the interface should be based on real world metaphor.
  - Disclose information in a progressive fashion.

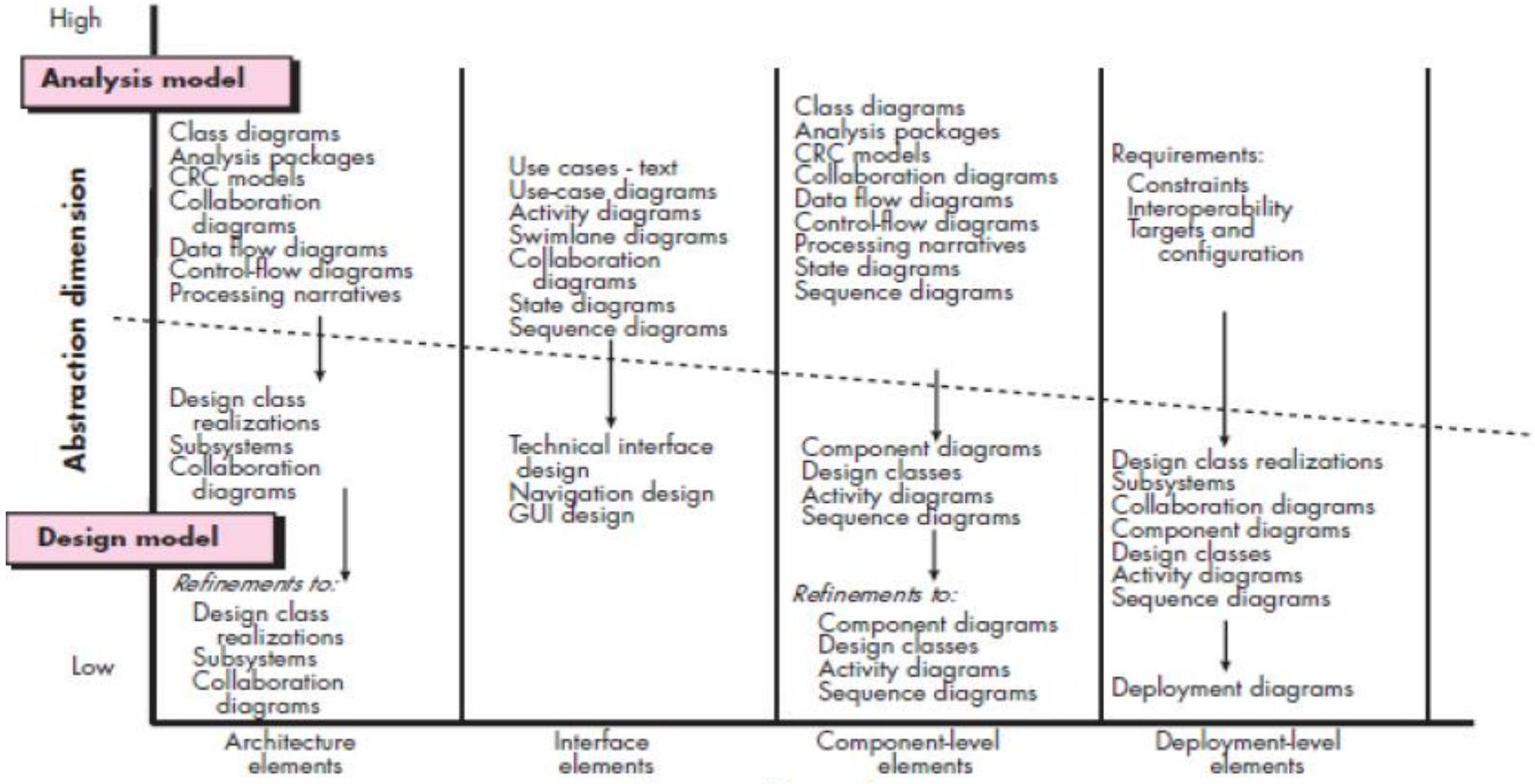
Golden Rules of UI Design

- Make the interface consistent: This means that
  - All visual information is organized according to a design standard that is maintained throughout all screen displays.
  - Input mechanisms are constrained to a limited set that are used consistently throughout the application.
  - Mechanisms for navigating from task to task are consistently defined and implemented.
  - Allow the user to put the current task into a meaningful content.
  - Maintain consistency across a family of applications.
  - If past interactive models have created user expectations, do not make changes unless there is a compelling reason to do so.

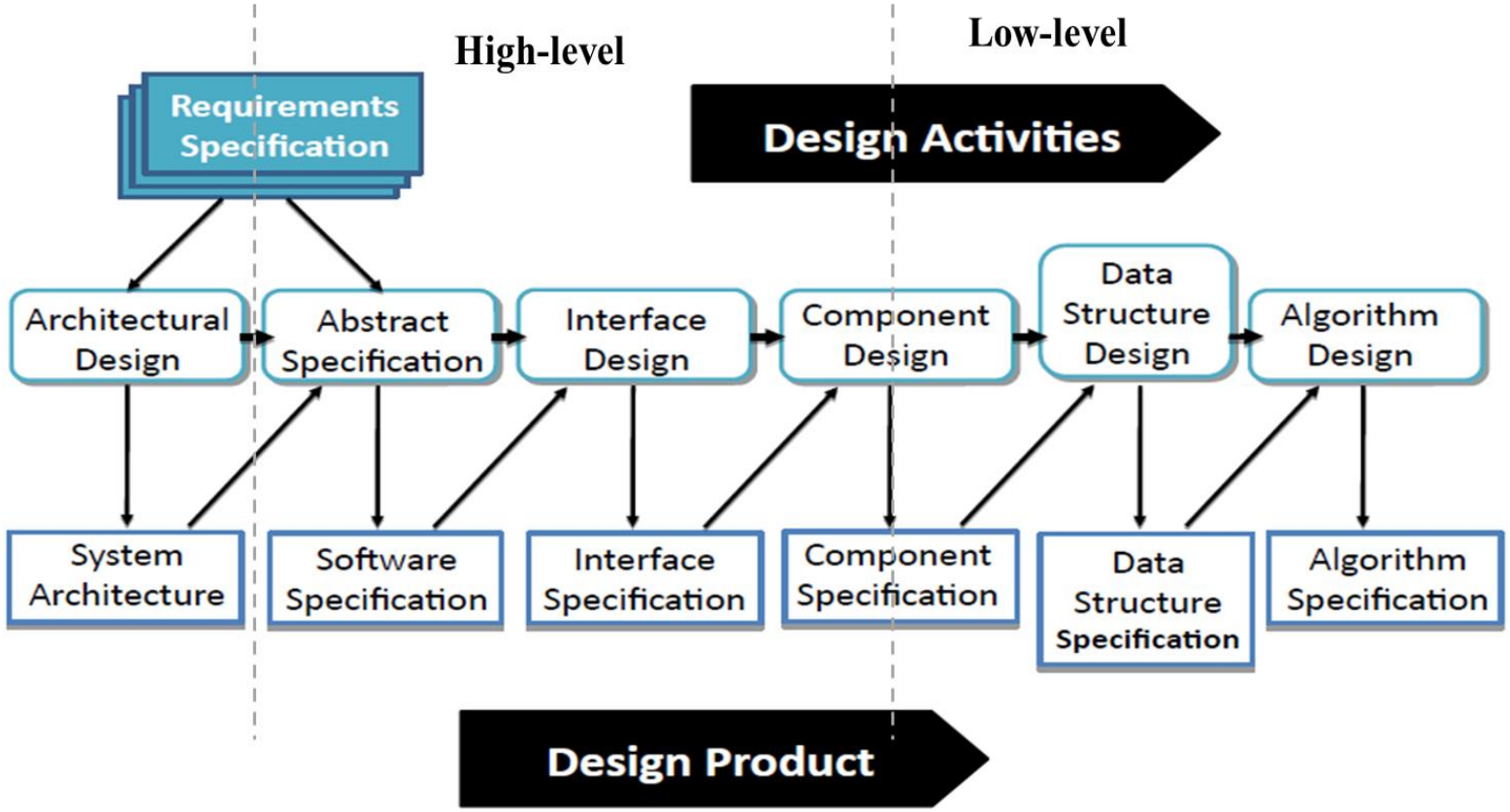
Component level design

- This is also called ***procedural design*** and occurs after data, architectural, and interface designs have been established.
- The ***3 layers of design must be translated into operational software***. To accomplish this, the design must be represented at a level of abstraction that is close to code.
- Component-level design establishes
  - The algorithmic detail required to manipulate data structures,
  - Effect communication between software components via their interfaces, and
  - Implement the processing algorithms allocated to each component.
- Graphical, tabular, or text-based notation is used to represent the design.

Process dimensions



Design process



High-level design,

- dividing the system into components, and

Low-level design

- choosing the data structures and algorithms for a single component.

Design Principles

Note : *They are often constrained by the existing hardware configuration, the implementation language, the existing file and data structures, and the existing organizational practices*

- The design process should not suffer from "tunnel vision" (*consider alternative approaches*).
- The design should be traceable to the analysis model.
- The design should not reinvent the wheel *(design to reuse).*
- The design should "minimize the intellectual distance" between the software and the problem as it exists in the real world (*easy to understand and use*).
- The design should exhibit uniformity and integration.
- The design should be structured to accommodate change.
- The design should be structured to degrade gently, even when aberrant data, events or operating conditions are encountered. e.g. *need arises for termination.*
- Design is not coding and coding is not design.
- The design should be assessed for quality as it is being created, not after the fact.
- The design should be reviewed to minimize conceptual (semantic) errors.

Design Concepts

- Abstraction: Permits one to concentrate on a problem at some level of generalization without regard to irrelevant low level details; use of abstraction also permits one to work with concepts and terms that are familiar in the problem environment without having to transform them to an unfamiliar structure.
- Refinement: One or several instructions of the given program are decomposed into more detailed instruction. This successive decomposition or refinement or specifications terminates when all instructions are expressed in terms of any underlying computer or programming language.
- Modularity: It is the single attribute of software that allows a program to be intellectually manageable. Software is divided into separately named and addressable components, often called modules, that are integrated to satisfy problem requirements.
  - » consistency in development, reduced development time, and flexibility.

Effective Modular Design

» **if the partitioned modules are separately solvable, modifiable as well as compliable.**

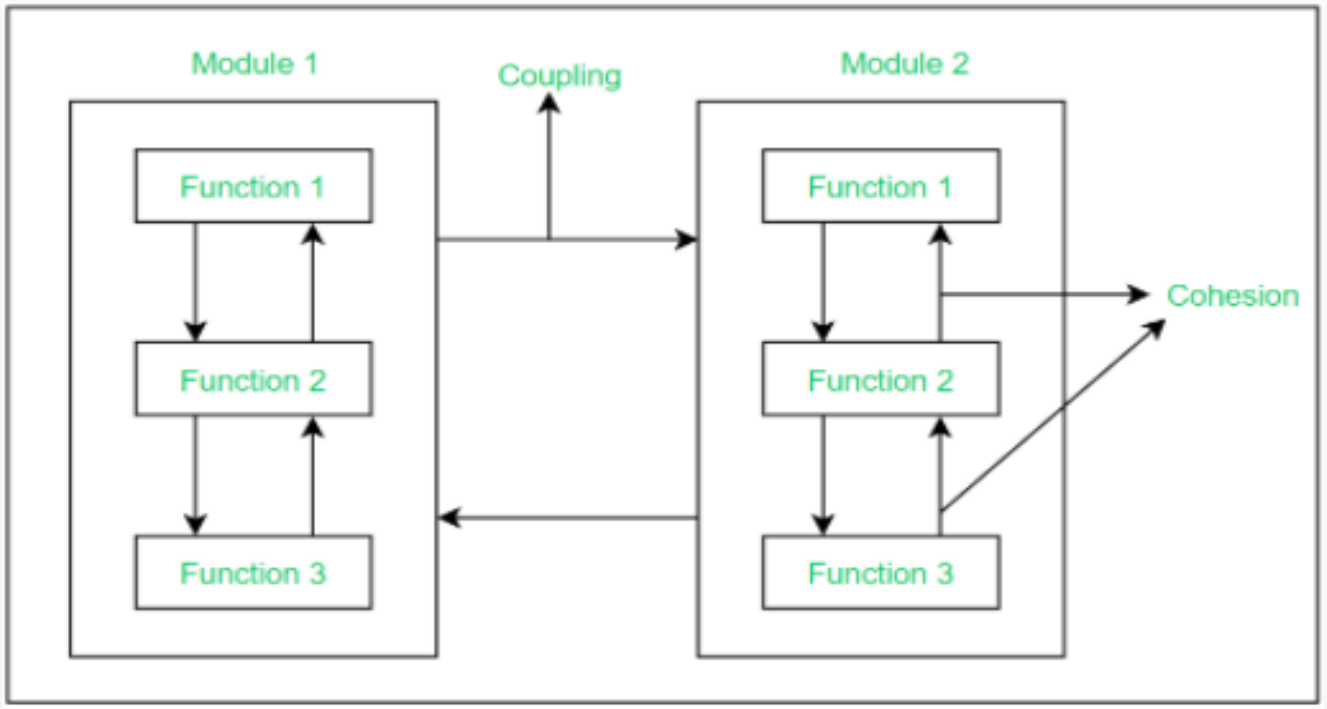
- Functional Independence: It is an outgrowth of modularity and the concepts of abstraction and information hiding. It means to design software in such a way that each module addresses a specific sub function of requirements and has a simple interface when viewed from other parts of the program structure.
- Cohesion: A cohesive module performs a single task within a software procedure, requiring little interaction with procedures being performed in other parts of the program. A cohesive model should ideally do just one thing.
- Coupling: It is a measure of interconnection among modules in a software structure. It depends of the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface.

Cont.

» *A good software design requires high cohesion and low coupling.*

Type of coupling low to high coupling:

1. Data Coupling
2. Stamp Coupling
3. Control Coupling
4. External Coupling
5. Common Coupling
6. Content Coupling



Type of cohesion high to low cohesion::

1. Functional cohesion
2. Sequential cohesion
3. Communicational cohesion
4. Procedural cohesion
5. Temporal cohesion
6. Logical cohesion
7. Co-incidental cohesion

1.High Cohesion: It means that components within a module or class should have a strong and focused relationship, with each part contributing to a specific, well-defined purpose.

2.Low Coupling: It implies that modules or classes should have minimal interdependencies or connections with other modules, reducing the ripple effect of changes and enhancing the system's flexibility and maintainability.

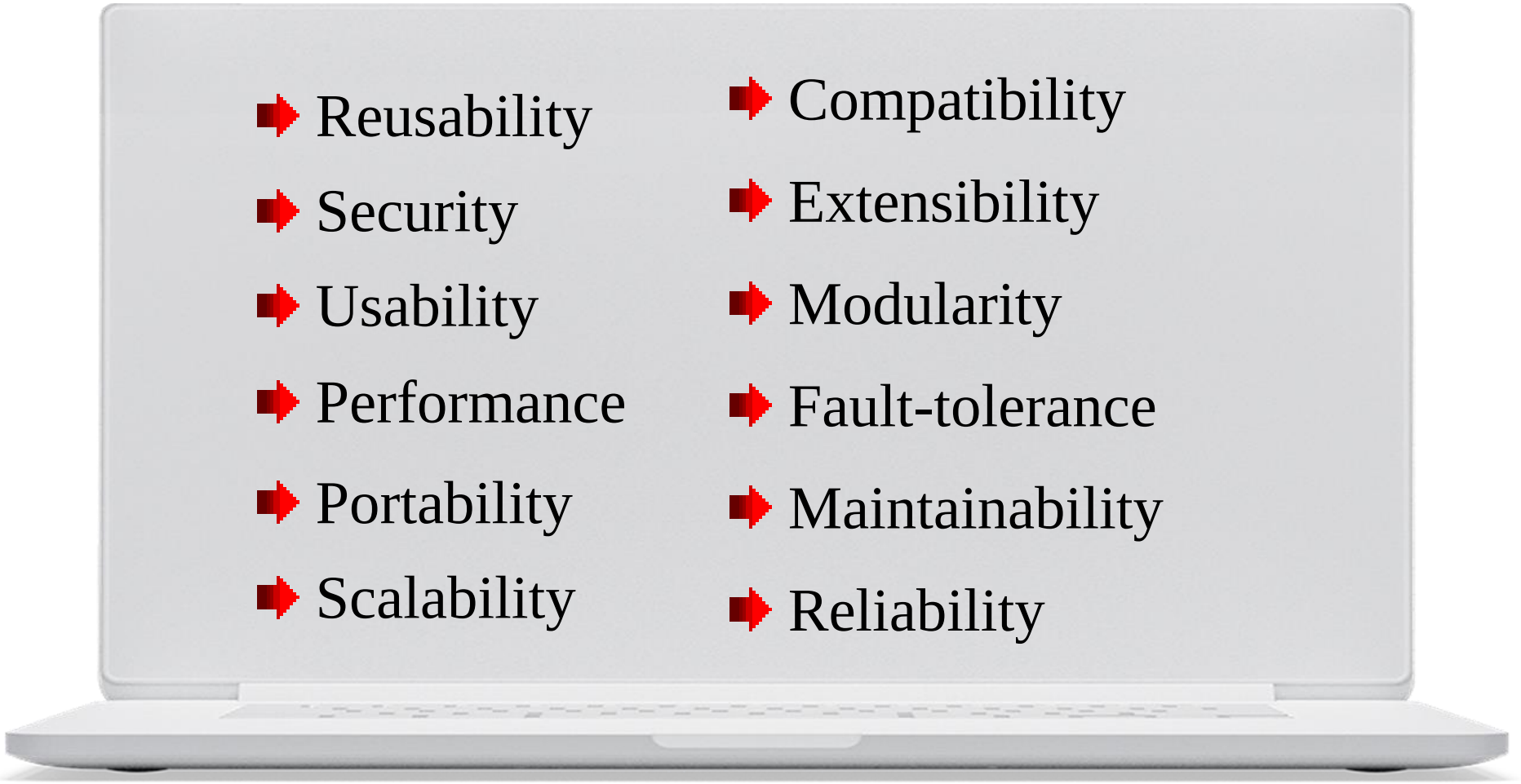
Design Concepts

- Pattern- a repeated form: means the repetition of a solution to a common recurring problem within a certain context.
- Refactoring- reconstruct something: means reconstructing the design to reduce complexity and simplify it without affecting the behavior or its functions.
- Software Architecture: Alludes to the overall structure of the software and the ways in which the structure provides conceptual integrity for the system.
- Control hierarchy: Also called program structure, it represents the organization of program components (modules) and implies the hierarchy of control.

Design Concepts

- Information hiding: Modules should be specified and designed so that information (procedure and data) contained within a module in inaccessible to other modules that have no need for such information.
- Structural partitioning: If the architectural style of the system is hierarchical the program structure can be partitioned either horizontally or vertically.
- Data structure: It is the representation of the logical relationship among individual elements of data. Because the structure of information will invariable affect the final procedural design, it is as important as program structure to the software architecture.
- Software procedure: Focuses on the processing details of each module individually.

Design considerations



The Design Document

- The overall scope of the design effort is described. Much of the information presented here is derived from the *Software Requirement Specification (SRS)*.
- Next, the data design is specified followed by the architectural design, interface design and components design.
- The *Design specification* contains a requirements cross specification.
- The first stage in the test documentation is also contained.
- *Design constraints* such as physical memory limitations or the necessity for a specialized external interface may be specified.
- *Special considerations* - high-speed processing or other factors may be specified.
- The final section contains supplementary data (algorithmic descriptions, tabular data etc.). It is also suggested to provide an appendix consisting of preliminary operations/manuals.

Hence SAD..

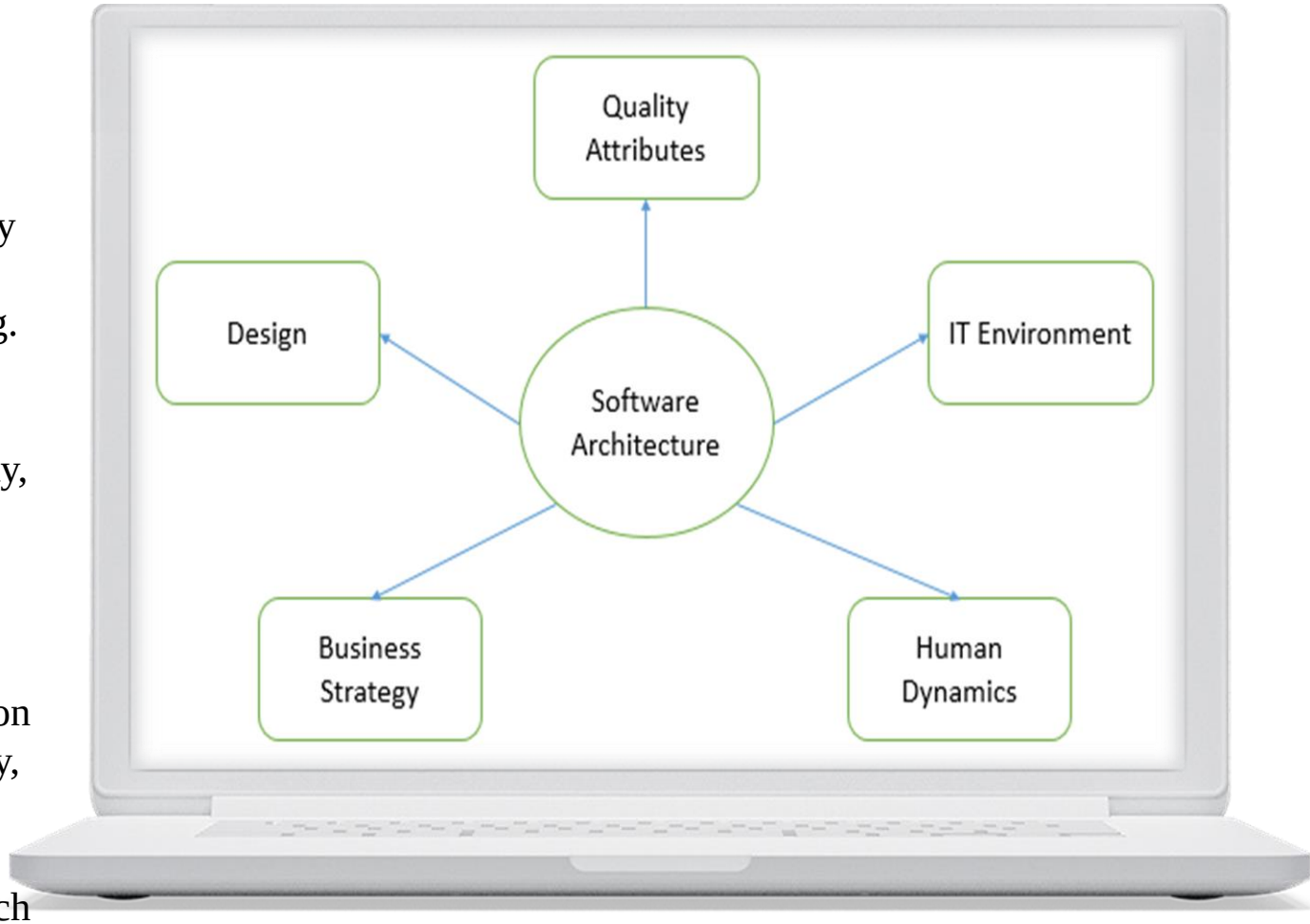
Software architecture and design includes several contributory factors such as :

1. Business Strategy: The overarching plan or approach a company uses to achieve its goals. it involves aligning technology decisions with the organization's long-term objectives and market positioning.

2. Quality Attributes: Characteristics of a software system that define its overall quality, such as performance, reliability, scalability, security, and maintainability. Architects consider these attributes when designing the system.

3. Human Dynamics: The interpersonal relationships, communication, and collaboration among team members working on a software project. It influences decision-making, team productivity, and the overall success of the project.

4. Design and IT Environment: The technological context in which the software will operate. This includes hardware, software platforms, tools, and existing systems, which impact architectural choices and system integration.



To be continued.—>



Quiz-1

1. What by meant software architecture and software design, and distinguish at least four basic difference among them?
2. How can we measure the quality of the software design?
3. List and explain at least four designs principles and four design concepts?
4. What constitutes effective requirement engineering?
5. *"The beginning of wisdom for a software engineer is to recognize the difference between getting a program to work, and getting it right."* As a software engineering student what do understand from the quote?

